

# Sun Avoidance

## *Component Design Document*

## 1 Description

Sun avoidance attitude guidance. Superimposes a constant rate slew from the current body attitude onto the input attitude reference, taking the long way around when the short slew would sweep a sensitive body axis across the Sun. The slew is planned on the first tick after a reset and fed forward at the configured rate until the reference is reached. Wraps the SunAvoidanceAlgorithm C++ algorithm via its C shim.

## 2 Requirements

No requirements have been specified for this component.

## 3 Design

### 3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 4
- **Number of Invokee Connectors** - 3
- **Number of Invoker Connectors** - 1
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - 2
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - *None*
- **Number of Data Dependencies** - 3
- **Number of Packets** - *None*

## 3.2 Diagram

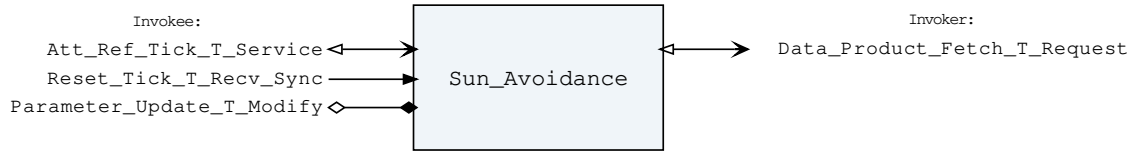


Figure 1: Sun Avoidance component diagram.

## 3.3 Connectors

Below are tables listing the component's connectors.

### 3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Sun Avoidance Invokee Connectors

| Name                      | Kind      | Type               | Return_Type | Count |
|---------------------------|-----------|--------------------|-------------|-------|
| Att_Ref_Tick_T_Service    | service   | Att_Ref_Tick.T     | Att_Ref.T   | 1     |
| Reset_Tick_T_Recv_Sync    | recv_sync | Tick.T             | -           | 1     |
| Parameter_Update_T_Modify | modify    | Parameter_Update.T | -           | 1     |

Connector Descriptions:

- **Att\_Ref\_Tick\_T\_Service** - Adjust the attitude reference in the argument away from the sun, at the tick in the argument, and return the result.
- **Reset\_Tick\_T\_Recv\_Sync** - Discard the planned slew so the next tick plans a new one from the current geometry. Must be called on any transition into the guidance mode that uses this component.
- **Parameter\_Update\_T\_Modify** - The parameter update connector.

### 3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Sun Avoidance Invoker Connectors

| Name                         | Kind    | Type                 | Return_Type           | Count |
|------------------------------|---------|----------------------|-----------------------|-------|
| Data_Product_Fetch_T_Request | request | Data_Product_Fetch.T | Data_Product_Return.T | 1     |

Connector Descriptions:

- **Data\_Product\_Fetch\_T\_Request** - Fetch a data product item from the database.

## 3.4 Interrupts

This component contains no interrupts.

### 3.5 Initialization

Below are details on how the component should be initialized in an assembly.

#### 3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

#### 3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

#### 3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 3: Sun Avoidance Set Id Bases Parameters

| Name              | Type                              |
|-------------------|-----------------------------------|
| Parameter_Id_Base | Parameter_Types.Parameter_Id_Base |

Parameter Descriptions:

- **Parameter\_Id\_Base** - The value at which the component's parameter identifiers begin.

#### 3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 4: Sun Avoidance Map Data Dependencies Parameters

| Name                            | Type                               |
|---------------------------------|------------------------------------|
| Spacecraft_Attitude_Id          | Data_Product_Types.Data_Product_Id |
| Spacecraft_Attitude_Stale_Limit | Ada.Real_Time.Time_Span            |
| Spacecraft_State_Id             | Data_Product_Types.Data_Product_Id |
| Spacecraft_State_Stale_Limit    | Ada.Real_Time.Time_Span            |
| Sun_State_Id                    | Data_Product_Types.Data_Product_Id |
| Sun_State_Stale_Limit           | Ada.Real_Time.Time_Span            |

Parameter Descriptions:

- **Spacecraft\_Attitude\_Id** - The external data product ID used to fetch the value for the 'Spacecraft\_Attitude' data dependency.
- **Spacecraft\_Attitude\_Stale\_Limit** - The stale limit used for the 'Spacecraft\_Attitude' data dependency. If the fetched data product is older than the given `Time_Span` then it is deemed stale and unfit for use by the component.
- **Spacecraft\_State\_Id** - The external data product ID used to fetch the value for the 'Spacecraft\_State' data dependency.

- **Spacecraft\_State\_Stale\_Limit** - The stale limit used for the 'Spacecraft\_State' data dependency. If the fetched data product is older than the given Time\_Span then it is deemed stale and unfit for use by the component.
- **Sun\_State\_Id** - The external data product ID used to fetch the value for the 'Sun\_State' data dependency.
- **Sun\_State\_Stale\_Limit** - The stale limit used for the 'Sun\_State' data dependency. If the fetched data product is older than the given Time\_Span then it is deemed stale and unfit for use by the component.

### 3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the sun avoidance algorithm with the default parameter values. The `init` subprogram takes no parameters.

## 3.6 Commands

The Sun Avoidance component has no commands.

## 3.7 Parameters

Parameters for the Sun Avoidance component.

Table 5: Sun Avoidance Parameters

| Local ID   | Parameter Name  | Type           | Default Value        |
|------------|-----------------|----------------|----------------------|
| 0x0000 (0) | Sensitive_Hat_B | Packed_F32x3.T | [0.0, 0.0, 1.0]      |
| 0x0001 (1) | Slew_Rate       | Packed_F32.T   | (Value=>0.017453292) |

Parameter Descriptions:

- **Sensitive\_Hat\_B** - [-] Body frame unit vector to keep away from the Sun during the slew. Must be finite and within 1e-3 of unit length.
- **Slew\_Rate** - [rad/s] Rate at which the slew toward the input reference is fed forward. Must be finite and greater than zero.

## 3.8 Events

The Sun Avoidance component has no events.

## 3.9 Data Products

The Sun Avoidance component has no data products.

## 3.10 Data Dependencies

Data dependencies for the Sun Avoidance component.

Table 6: Sun Avoidance Data Dependencies

| Data Dependency Name | Type |
|----------------------|------|
|----------------------|------|

|                     |                   |
|---------------------|-------------------|
| Spacecraft_Attitude | Nav_Att_Output.T  |
| Spacecraft_State    | Cartesian_State.T |
| Sun_State           | Cartesian_State.T |

Data Dependency Descriptions:

- **Spacecraft\_Attitude** - Spacecraft attitude estimate. Only the MRP attitude of the body relative to inertial is used.
- **Spacecraft\_State** - [m, m/s] Spacecraft inertial position and velocity. Only the position is used.
- **Sun\_State** - [m, m/s] Sun inertial position and velocity. Only the position is used. An all-zero position means no Sun information, and the input reference then passes through unchanged.

### 3.11 Packets

The Sun Avoidance component has no packets.

### 3.12 Faults

The Sun Avoidance component has no faults.

## 4 Unit Tests

The following section describes the unit test suites written to test the component.

### 4.1 *Sun\_Avoidance\_Tests* Test Suite

This is a unit test suite for the Sun Avoidance component

Test Descriptions:

- **Test** - Run a slew with the Sun clear of the swept arc to ensure the Ada to C to C++ integration is sound.
- **Test\_Long\_Way\_Around** - Check that a Sun inside the swept arc makes the slew go the long way around.
- **Test\_Pass\_Through\_Without\_Sun** - Check that an all-zero Sun position passes the input reference through unchanged.
- **Test\_Invalid\_Parameter** - Ensure a staged configuration the algorithm would reject is refused at validation.
- **Test\_Invalid\_Data\_Dependency** - Ensure a data dependency with the wrong identifier is treated as a wiring defect and fails the tick's assertion.

## 5 Appendix

### 5.1 Preamble

This component contains no preamble code.

## 5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

### Att\_Ref.T:

Table 7: Att\_Ref Packed Record : 288 bits

| Name        | Type           | Range | Size (Bits) | Start Bit | End Bit |
|-------------|----------------|-------|-------------|-----------|---------|
| Sigma_Rn    | Packed_F32x3.T | -     | 96          | 0         | 95      |
| Omega_Rn_N  | Packed_F32x3.T | -     | 96          | 96        | 191     |
| Domega_Rn_N | Packed_F32x3.T | -     | 96          | 192       | 287     |

Field Descriptions:

- **Sigma\_Rn** - *No description provided.*
- **Omega\_Rn\_N** - *No description provided.*
- **Domega\_Rn\_N** - *No description provided.*

### Att\_Ref\_Tick.T:

The tick a guidance stage runs on and the attitude reference it starts from. The component that chains the guidance stages hands this to a stage and receives the stage's reference back.

Table 8: Att\_Ref\_Tick Packed Record : 368 bits

| Name         | Type      | Range | Size (Bits) | Start Bit | End Bit |
|--------------|-----------|-------|-------------|-----------|---------|
| Current_Tick | Tick.T    | -     | 80          | 0         | 79      |
| Reference    | Att_Ref.T | -     | 288         | 80        | 367     |

Field Descriptions:

- **Current\_Tick** - The tick the stage runs on.
- **Reference** - The attitude reference the stage starts from.

### Cartesian\_State.T:

Cartesian state containing position and velocity vectors.

Table 9: Cartesian\_State Packed Record : 384 bits

| Name     | Type           | Range | Size (Bits) | Start Bit | End Bit |
|----------|----------------|-------|-------------|-----------|---------|
| Position | Packed_F64x3.T | -     | 192         | 0         | 191     |
| Velocity | Packed_F64x3.T | -     | 192         | 192       | 383     |

Field Descriptions:

- **Position** - [m] Position vector
- **Velocity** - [m/s] Velocity vector

### Data\_Product.T:

Generic data product packet for holding arbitrary data types

Table 10: Data\_Product Packed Record : 1096 bits (*maximum*)

| Name   | Type                                        | Range | Size (Bits) | Start Bit | End Bit | Variable Length      |
|--------|---------------------------------------------|-------|-------------|-----------|---------|----------------------|
| Header | Data_Product_Header.T                       | -     | 72          | 0         | 71      | -                    |
| Buffer | Data_Product_Types.Data_Product_Buffer_Type | -     | 1024        | 72        | 1095    | Header.Buffer_Length |

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

### Data\_Product\_Fetch.T:

A packed record which holds information for a data product request.

Table 11: Data\_Product\_Fetch Packed Record : 16 bits

| Name | Type                               | Range      | Size (Bits) | Start Bit | End Bit |
|------|------------------------------------|------------|-------------|-----------|---------|
| Id   | Data_Product_Types.Data_Product_Id | 0 to 65535 | 16          | 0         | 15      |

Field Descriptions:

- **Id** - The data product identifier

### Data\_Product\_Header.T:

Generic data\_product packet for holding arbitrary data\_product types

Table 12: Data\_Product\_Header Packed Record : 72 bits

| Name          | Type                                               | Range      | Size (Bits) | Start Bit | End Bit |
|---------------|----------------------------------------------------|------------|-------------|-----------|---------|
| Time          | Sys_Time.T                                         | -          | 48          | 0         | 47      |
| Id            | Data_Product_Types.Data_Product_Id                 | 0 to 65535 | 16          | 48        | 63      |
| Buffer_Length | Data_Product_Types.Data_Product_Buffer_Length_Type | 0 to 128   | 8           | 64        | 71      |

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer\_Length** - The number of bytes used in the data product buffer

### Data\_Product\_Return.T:

This record holds data returned from a data product fetch request.

Table 13: Data\_Product\_Return Packed Record : 1104 bits (*maximum*)

| Name             | Type                              | Range                                                      | Size (Bits) | Start Bit | End Bit | Variable Length |
|------------------|-----------------------------------|------------------------------------------------------------|-------------|-----------|---------|-----------------|
| The_Status       | Data_Product_Enums.Fetch_Status.E | 0 => Success<br>1 => Not_Available<br>2 => Id_Out_Of_Range | 8           | 0         | 7       | –               |
| The_Data_Product | Data_Product.T                    | -                                                          | 1096        | 8         | 1103    | –               |

Field Descriptions:

- **The\_Status** - A status relating whether or not the data product fetch was successful or not.
- **The\_Data\_Product** - The data product item returned.

### Nav\_Att\_Output.T:

Table 14: Nav\_Att\_Output Packed Record : 352 bits

| Name            | Type           | Range       | Size (Bits) | Start Bit | End Bit |
|-----------------|----------------|-------------|-------------|-----------|---------|
| Time_Tag        | Long_Float     | -inf to inf | 64          | 0         | 63      |
| Sigma_Bn        | Packed_F32x3.T | -           | 96          | 64        | 159     |
| Omega_Bn_B      | Packed_F32x3.T | -           | 96          | 160       | 255     |
| Veh_Sun_Pnt_Bdy | Packed_F32x3.T | -           | 96          | 256       | 351     |

Field Descriptions:

- **Time\_Tag** - [s] Time tag of navigation solution
- **Sigma\_Bn** - MRP attitude of body w.r.t. inertial frame
- **Omega\_Bn\_B** - [rad/s] Angular velocity of body w.r.t. inertial, body frame components
- **Veh\_Sun\_Pnt\_Bdy** - Sun direction unit vector in body frame

### Packed\_F32.T:

Single component record for holding packed 32-bit floating point number.

Table 15: Packed\_F32 Packed Record : 32 bits

| Name  | Type        | Range                       | Size (Bits) | Start Bit | End Bit |
|-------|-------------|-----------------------------|-------------|-----------|---------|
| Value | Short_Float | -3.40282e+38 to 3.40282e+38 | 32          | 0         | 31      |

Field Descriptions:

- **Value** - The 32-bit floating point number.

### Packed\_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 16: Packed\_F32x3 Packed Array : 96 bits

| Type               | Range                       | Element Size (Bits) | Length | Total Size (Bits) |
|--------------------|-----------------------------|---------------------|--------|-------------------|
| <b>Short_Float</b> | -3.40282e+38 to 3.40282e+38 | 32                  | 3      | 96                |

### Packed\_F64x3.T:

Packed 3 element vector of 64-bit floats.

Table 17: Packed\_F64x3 Packed Array : 192 bits

| Type              | Range       | Element Size (Bits) | Length | Total Size (Bits) |
|-------------------|-------------|---------------------|--------|-------------------|
| <b>Long_Float</b> | -inf to inf | 64                  | 3      | 192               |

### Parameter.T:

Generic parameter packet for holding a generic parameter

Table 18: Parameter Packed Record : 792 bits (*maximum*)

| Name   | Type                                      | Range | Size (Bits) | Start Bit | End Bit | Variable Length      |
|--------|-------------------------------------------|-------|-------------|-----------|---------|----------------------|
| Header | Parameter_Header.T                        | -     | 24          | 0         | 23      | -                    |
| Buffer | Parameter_Types.<br>Parameter_Buffer_Type | -     | 768         | 24        | 791     | Header.Buffer_Length |

Field Descriptions:

- **Header** - The parameter header
- **Buffer** - A buffer that contains the parameter type

### Parameter\_Header.T:

Generic parameter header for holding arbitrary parameters

Table 19: Parameter\_Header Packed Record : 24 bits

| Name          | Type                                             | Range      | Size (Bits) | Start Bit | End Bit |
|---------------|--------------------------------------------------|------------|-------------|-----------|---------|
| Id            | Parameter_Types.<br>Parameter_Id                 | 0 to 65535 | 16          | 0         | 15      |
| Buffer_Length | Parameter_Types.<br>Parameter_Buffer_Length_Type | 0 to 96    | 8           | 16        | 23      |

Field Descriptions:

- **Id** - The parameter identifier
- **Buffer\_Length** - The number of bytes used in the parameter type buffer

### Parameter\_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 20: Parameter\_Update Packed Record : 824 bits (*maximum*)

| Name      | Type                                           | Range                                                                       | Size (Bits) | Start Bit | End Bit | Variable Length |
|-----------|------------------------------------------------|-----------------------------------------------------------------------------|-------------|-----------|---------|-----------------|
| Table_Id  | Parameter_Types.<br>Parameter_Table_Id         | 0 to 65535                                                                  | 16          | 0         | 15      | –               |
| Operation | Parameter_Enums.<br>Parameter_Operation_Type.E | 0 => Stage<br>1 => Update<br>2 => Fetch<br>3 => Validate                    | 8           | 16        | 23      | –               |
| Status    | Parameter_Enums.<br>Parameter_Update_Status.E  | 0 => Success<br>1 => Id_Error<br>2 => Validation_Error<br>3 => Length_Error | 8           | 24        | 31      | –               |
| Param     | Parameter.T                                    | –                                                                           | 792         | 32        | 823     | –               |

Field Descriptions:

- **Table\_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

### Sys\_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 21: Sys\_Time Packed Record : 48 bits

| Name       | Type                       | Range           | Size (Bits) | Start Bit | End Bit |
|------------|----------------------------|-----------------|-------------|-----------|---------|
| Seconds    | Interfaces.<br>Unsigned_32 | 0 to 4294967295 | 32          | 0         | 31      |
| Subseconds | Interfaces.<br>Unsigned_16 | 0 to 65535      | 16          | 32        | 47      |

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of  $1/(2^{16})$  sub-seconds.

### Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 22: Tick Packed Record : 80 bits

| Name  | Type                       | Range           | Size (Bits) | Start Bit | End Bit |
|-------|----------------------------|-----------------|-------------|-----------|---------|
| Time  | Sys_Time.T                 | -               | 48          | 0         | 47      |
| Count | Interfaces.<br>Unsigned_32 | 0 to 4294967295 | 32          | 48        | 79      |

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

### 5.3 Enumerations

The following section outlines any enumerations used in the component.

#### Data\_Product\_Enums.Fetch\_Status.E:

This status denotes whether a data product fetch was successful.

Table 23: Fetch\_Status Literals:

| Name            | Value | Description                                           |
|-----------------|-------|-------------------------------------------------------|
| Success         | 0     | The data product was returned successfully.           |
| Not_Available   | 1     | No data product is yet available for the provided id. |
| Id_Out_Of_Range | 2     | The data product id was out of range.                 |

#### Parameter\_Enums.Parameter\_Operation\_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 24: Parameter\_Operation\_Type Literals:

| Name     | Value | Description                                                       |
|----------|-------|-------------------------------------------------------------------|
| Stage    | 0     | Stage the parameter.                                              |
| Update   | 1     | All parameters are staged, it is ok to update all parameters now. |
| Fetch    | 2     | Fetch the parameter.                                              |
| Validate | 3     | Validate all the parameters.                                      |

#### Parameter\_Enums.Parameter\_Update\_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 25: Parameter\_Update\_Status Literals:

| Name             | Value | Description                                       |
|------------------|-------|---------------------------------------------------|
| Success          | 0     | Parameter was successfully staged.                |
| Id_Error         | 1     | Parameter id was not valid.                       |
| Validation_Error | 2     | Parameter values were not successfully validated. |
| Length_Error     | 3     | Parameter length was not correct.                 |